Development Plan
DAIIM

Team 6,
Robert Lam
Nathan Hum
Aidan Lao
Keira Laskoski

Table 1: Revision History

Date	**Developer(s)**	**Change**
September 27	Nathan	Changed project decomposition and scheduling, changed GitHub Projects usage, changed coding standards, changed Appendix – Team Charter
September 27	Aidan	Added sections 5 to 7 of the development plan regarding Team communication plan, roles, and workflow.
September 27	Keira	Added section 13, the reflection, and part of the team charter to the report.
September 28	Robert	Added sections 1-4 and 9-10
September 28	Keira	Revised section 13
September 28	Nathan	Added section 11
September 28	All	Reviewed document and fixed formatting
October 8	Aidan	Refined section 9 POC success criteria to be less vague by defining concrete accuracy, malicious actor, and reward thresholds
October 26	Keira	Updated GenAI Disclosure to add more information regarding brainstorming uses
October 27	Aidan	Expanded the workflow plan with issue types, Kanban board usage, commit types, and pull request steps; fixed formatting and grammar (TA feedback)
November 4	Keira	Added and assigned specific team roles

This document outlines the processes, tools, and practices that the team will use to plan, develop, and manage the DAIIM project. It describes the team's approach to communication, collaboration, project scheduling, workflow, and software development, as well as the technologies and coding standards that will guide implementation. The plan also identifies the project's proof-of-concept strategy and describes how generative AI and other development tools will be incorporated into the development process.

1 Confidential Information?

This project does not contain any confidential or protected information.

2 IP to Protect

The team does not have any project-specific intellectual property that requires protection through an IP agreement. The DAIIM source code will be released as open-source software under the license described in the copyright license section below.

3 Copyright License

This project, including all source code and documentation, will be under the MIT license. The license can be found in the project repository.

4 Team Meeting Plan

Table 2: Team and Supervisor Meeting Schedule

Meeting	**Location**	**Frequency**	**Time**
Team meeting	McMaster campus	Weekly	Wednesday 1:30 pm
Supervisor meeting	Microsoft Teams	Biweekly	TBD

All meetings will be recorded in a GitHub issue outlining the date, attendance and agenda. Each meeting will have a chair who is responsible for creating the agenda and sharing it with others prior to the meeting. If possible, the agenda should be shared 12 hours in advance to give team members a chance to review and propose different topics. For each meeting, the chair will ask for a volunteer from the team to record meeting minutes. If no one volunteers, the note-taking role will rotate among team members. Meeting minutes and decisions made during the meeting will be recorded as a comment on the meeting issue. Ad

hoc meetings may also be scheduled as necessary for urgent decisions, issue resolution, or preparation for a deliverable.

5 Team Communication Plan

The team will use a Microsoft Teams group chat for regular communication. The team also has a shared Word document for collaborative editing of documents and planning during meetings. GitHub issues will be used to track each meeting and any associated agendas/minutes. Communication with the supervisor will be done through email and Microsoft Teams. Every email with the supervisor will be sent to all team members. The team will meet regularly with the supervisor, subject to the supervisor's availability.

GitHub issues will be the primary method of communication for development tasks. Each issue will represent a development task and will be assigned to a team member; the team member will be responsible for updating the issue with their progress. The issues will be associated with a milestone representing the deliverable, and will be placed appropriately in the Kanban project according to its status (see Section 7.2). Feedback is expected from at least one other team member for each issue, through either a comment on the GitHub issue or reviewed in a meeting or group chat.

6 Team Member Roles

- **Agenda creator (Keira, Robert)**: The agenda creator is responsible for creating the agenda for a meeting, and posting it to the meeting issue. The agenda creator will use the template for team meetings located in the repo, containing the agenda and an attendance tracker.
- **Notetaker (All)**: The notetaker is responsible for taking notes during a meeting, and posting the meeting minutes to the meeting issue. The meeting minutes will be posted as a comment under the meeting issue. Multiple people may contribute to the notetaker role during a meeting, especially if there are multiple topics / individual updates from each person.
- **Pull request reviewer (All)**: The pull request reviewer role is a role that is assigned to a team member for each pull request. The pull request reviewer is responsible for reviewing the code changes in the pull request, providing feedback, and approving or requesting changes to the pull request. Every team member is expected to review pull requests, but the pull request reviewer is the primary reviewer for that pull request. They are in charge of making the final decision on approving the request.
- **Developer (All)**: The developer role is a role assigned to a team member for each development task. The developer is responsible for implementing

the task, and updating the associated issue with their progress. Every team member is expected to take on this role and contribute to development tasks as necessary, but the developer is the primary person responsible for completing a given task. The development will be supported by all members. Leads are assigned and listed as follows:

  - **Frontend and UX Lead (Keira)**: Build the web application and wallet integration for submitting requests, authorizing transactions, and viewing results. Lead the cognitive walkthrough and usability testing.
  - **Backend Provider SDK Lead (Aidan)**: Build the inference provider client that listens for requests, runs models, and submits predictions. Develop the simulated providers.
  - **Smart Contract and Blackchain Lead (Robert)**: Design and implement the marketplace smart contracts for task definitions, inference requests, provider registration, payments, and settlement. Manage deployment to the test network and on-chain record.
  - **Aggregation and Reputation Lead (Nathan)**: Design and implement the prediction aggregation algorithm and the provider reputation and reward mechanisms.
  - **Test Lead (All)**: The test developer will be in charge of writing tests for components of the system. Components will be assigned to all members of the team to ensure testing covers all part of the system.

- **Document Verifier (All)**: The document verifier will review documents and check them against rubrics and checklists. This role will be assigned to one member for every document, and will change throughout the duration of the project.

7 Workflow Plan

The following workflow plan can be used as a contributor's guide to the development process. Every piece of work follows the same path: an issue is created and labelled, its card moves across the Kanban board, the work is done on a branch, and it is merged through a reviewed pull request.

7.1 Issues and Issue Types

Each deliverable (and sub-deliverable if needed) will have a GitHub milestone. Every task needed to complete a deliverable will be tracked as a GitHub issue under that milestone. Each issue must have a type label, an assignee, a milestone, and a target date before work on it starts. Issues will be created from the

templates in the repository where one exists, so that the information needed for each type of issue is always included. Table 3 lists the issue types.

Table 3: Issue Types

Type	Label	Used for and required content
Feature	`enhancement`	New functionality in the code. Describes the expected behaviour and the requirements (e.g., SRS functional requirements) it implements. Closed by a merged pull request.
Bug	`bug`	Defects in existing code or the build pipeline. Includes steps to reproduce, expected behaviour, and actual behaviour. Closed by a merged pull request that fixes the bug and, where possible, adds a test for it.
Documentation	`documentation`	Writing or revising a section of a deliverable. Names the document and section(s) affected. Closed by a merged pull request.
Feedback	`review`	Feedback from the TA, supervisor, or a peer review team. Created with the peer review template and titled `<Source> - <Document> - <summary>` (e.g., "TA Feedback - Dev Plan - Formatting issues"). Closed by a merged pull request that addresses it, or by a comment explaining why no change was made.
Meeting	`meeting`	Team, supervisor, TA, and lecture meetings. Created with the matching meeting template, which holds the agenda and attendance. Closed once the minutes have been posted as a comment.

7.2 Kanban Board Usage

Issues are tracked on the team's GitHub Kanban board, which has the columns described in Table 4. The assignee is responsible for moving their own cards. The board will be reviewed at each weekly team meeting, where backlog issues are assigned and moved to Ready and each member gives an update on their In Progress cards.

Table 4: Kanban Board Columns

Column	A card is in this column when
Backlog	The issue has been created but is not yet planned or assigned.
Ready	The issue has a label, assignee, milestone, and target date, and can be started.
In Progress	The assignee has started work and created a branch for the issue.
In Review	A pull request for the issue is open and a reviewer has been assigned. The reviewer responds within 48 hours, as set out in the team charter.
Done	The pull request has been merged, or the issue has been closed without code changes (e.g., a meeting issue with its minutes posted).

7.3 Branches and Commits

Branch naming convention: All work will be done on a new branch named `<branch-type>/<branch-name>`, where the branch name is a short kebab-case description of the work. For example, a branch for the issue "Implement user login" would be named `feat/implement-user-login`. Typically, a branch will be created for each issue, but a branch may close several issues if they are related and can be completed together. The branch types are the same as the first four commit types in Table 5: `feat`, `fix`, `docs`, and `test`.

Commit naming convention: Each commit message will follow the format `<commit-type>: <commit-message>`, based on the Conventional Commits specification. The message is written in lowercase and in the imperative mood, and describes what the commit changes. For example, a commit for the issue "Implement user login" would be named `feat: implement user login`. A commit that completes an issue references it in its body (e.g., `Closes #42`). Commits made automatically by the CI pipeline, such as regenerated PDFs, are exempt from this convention.

Table 5: Commit Types

Type	Used for
`feat`	A new feature or new behaviour in the code.
`fix`	A bug fix, or a correction made in response to feedback.
`docs`	Changes to documentation, including the course deliverables.
`test`	Adding or updating tests.
`refactor`	Code changes that neither fix a bug nor add a feature.
`style`	Formatting-only changes, such as running `Prettier` or `rustfmt`.
`ci`	Changes to the GitHub Actions workflows or build scripts.
`chore`	Maintenance that does not fit another type, such as dependency updates.

7.4 Pull Requests and Code Review

A pull request will be created once the work for an issue is done. Its title follows the commit naming convention, and its description summarizes the changes and links the issue(s) it resolves with `Closes #<issue-number>`, so that the issue closes when the pull request is merged. The author assigns a primary reviewer and moves the card to In Review. Before requesting review, the author merges the latest `main` into the branch and resolves any conflicts. The reviewer approves the pull request or requests changes, and gives at least one point of constructive feedback, as described in the team charter.

7.5 Merging Strategy

Developers will merge their changes into the main branch through pull requests. A pull request must be approved by at least one other team member, and the CI checks (including the LaTeX build) must pass, before it can be merged into the main branch. Pull requests will be merged with a merge commit, so that individual commits are preserved in the history.

8 Project Decomposition and Scheduling

The project is organized around key milestones and overarching stages (refer to Table 6), progressing from initial project planning and requirements analysis through design, implementation, verification, and final demonstration. Each stage contains defined deliverables and deadlines to provide the team with a clear progression throughout the project. The schedule will remain flexible where necessary, allowing the team to adjust tasks and responsibilities based on project needs, progress, and individual strengths.

Stage	Milestone	Deadline
Planning	Project selection	W2
Planning	Problem Statement + PoC + Development Plan + Team Charter	W4
Requirements	SRS + Hazard Analysis	W6
Verification	V&V Plan	W8
Design	Design Document	W10
Design	PoC Demo	W11
Implementation	Design Document Rev 0	W16
Implementation	Module Implementation	W16
Implementation	Rev 0 Demo	W18
Finalization	Final Demo + Documentation	W21
Finalization	V&V Report + CD	W22
Finalization	Final Demos	W23
Expo	Expo Materials	W25
Expo	Capstone EXPO	W26

Table 6: Project Milestones and Schedule

8.1 GitHub Project Usage

The project board will follow a Kanban-style workflow with columns for Backlog, Ready, In Progress, In Review, and Done. The project will be organized into milestones representing the overarching deliverables required throughout each stage of the project. For example, the initial stage contains milestones for the Project Statement and Development Plan, with individual tasks associated with each milestone. Each task will be represented as a card on the board and assigned to the appropriate team member. Cards will move through the Kanban columns as work progresses, as described in Section 7.2, allowing the team to track the status of individual tasks and overall milestone completion.

9 Proof of Concept Demonstration Plan

9.1 Project Risks

One main risk is designing an aggregation algorithm that combines predictions from multiple inference providers into an accurate final result. The algorithm must handle disagreements between providers and reduce the influence of dishonest or low-quality predictions. This is difficult because, for a given task, the network may not have a source of *ground truth* against which to compare the predictions. There is also a cold-start problem where initially there may not be enough trusted providers in the network to correctly evaluate one another.

Another risk is designing the incentive system and token economics so that the network is sufficiently decentralized and is resistant to attacks. The system must

be fair and accessible for participants so that they are encouraged to take part in the network. The system must also be economically sustainable and ensure that the distribution of rewards does not create incentives to exploit the system or reduce the quality of the aggregated results.

Collusion attacks pose a significant risk to the system. A collusion attack occurs when multiple participants, such as inference providers, coordinate their actions to manipulate the aggregation algorithm, reputations, and reward distribution. For example, providers could submit similar inaccurate predictions to make their results appear correct and receive rewards. This is difficult to detect when there is no ground truth and when the colluding providers make up a large portion of the network.

9.2 POC Demonstration Plan

The proof of concept plan is to create a simulation of the DAIIM network that is completely off-chain. The simulation will execute a sequence of rounds in which an inference task for a regression problem is executed by the network. Several types of actors will participate in the network:

1. Honest: attempt to maximize rewards by providing accurate predictions consistently (with some noise);
2. Learners: gradually improve their predictions over time;
3. Malicious: intentionally provide bad predictions;
4. Colluding: a group of providers working together to skew the network.

The simulation will start with a simple Machine Learning (ML) task (e.g., predict y, where $y = mx + b$), and each round some input data will be randomly generated. The different actors will be given some algorithm (good or bad algorithm depending on the actor) to determine their predictions given the input. The actors will submit their predictions, and the aggregation algorithm will determine the aggregated result. Actors' reputations will be updated and rewards will be distributed after each round. The ground truth will be known to the testers and used only to evaluate the results. It will not be available to the aggregation algorithm, reputation updates, or reward distribution.

Another layer of this simulation is to economically model the system. We will assign providers a cost for running their model, transaction fees for submitting predictions on-chain, and economic rewards or penalties for participating. These values will be calibrated relative to the costs and payments that we expect a provider to realistically encounter when performing an inference task. This will ensure that the simulation reflects realistic participation decisions rather than relying on arbitrary values. We will also vary these values to measure how sensitive participation and network performance are to changes in costs, fees, rewards, and penalties.

We will test various combinations of actors to evaluate the limits of the system. These scenarios will vary the proportions of different actors, as well as the number of providers in the network. These results will allow us to identify conditions under which the system remains reliable and economically viable, as well as conditions under which malicious or colluding providers can significantly influence the network.

9.3 Consequences of Failure

If the proof of concept fails and one or more of the identified risks are not shown to be sufficiently mitigated, this would indicate that the current system is not able to provide accurate aggregated predictions, is not economically attractive to participants or is vulnerable to collusion attacks. This would require the team to re-evaluate the scope of the project and/or the design of the system. If the proof of concept is unsuccessful due to a lack of developer resources and/or technical expertise, then the team will meet with the supervisor to discuss shrinking the scope. Otherwise, if the cause of failure is due to poor system design then the team will meet with the supervisor to discuss alternative approaches to consensus, incentive design, and aggregation algorithms.

9.4 Success Criteria and Failure Response

This proof of concept will be successful and show that the stated risks can be overcome if:

- The system provides accurate aggregated results that improve over time even with some limited number of malicious actors. We define accurate as the aggregated result being within a certain threshold (roughly 90% of aggregated predictions match the ground truth). For this POC, we assume that the aggregated result will improve even with up to 10% malicious actors in the network. The exact threshold for these percentages will be determined during the POC implementation and testing.
- It is more profitable to act honestly and attempt to provide good predictions than to be malicious. Good predictions are those that are close to the ground truth (low relative error ¡ 10%). These good predictions should be rewarded with a non-zero token amount. The exact threshold for this percentage will be determined during the POC implementation and testing.
- The network can be cold-started with the assumption that initially there are a majority of honest providers.

If the proof of concept is not able to demonstrate that these risks can be addressed then the project's assumptions and scope may need to be changed. Before proceeding to full implementation, the team will review the findings with the supervisor and decide whether to revise the design, broaden the threat conditions, or reconsider the project's feasibility.

10 Expected Technology

This section describes the technologies that are expected to support the implementation of DAIIM. Note that these choices are preliminary and subject to change later in the project.

10.1 Core Implementation Stack

DAIIM will use Solana as the blockchain network for recording transactions and executing on-chain logic. During development, the team will use Solana Devnet, a separate network for testing programs and transactions without affecting Solana Mainnet. We will request free test SOL from a Devnet faucet to pay for transaction fees. Since Devnet tokens have no real-world value, the team can develop and test DAIIM without using real money before deploying to Mainnet.

Solana programs, or smart contracts, will be written in Rust. The conventional off-chain backend will use Node.js with TypeScript and PostgreSQL for data storage. The user-facing interface will be developed with React.js and Vite. Python may also be used to write simple machine learning models for integration testing purposes.

10.2 Libraries and External Components

The expected external libraries and components are:

- *[Anchor]*: A development framework for building, deploying and testing Solana programs in Rust.
- *[MetaMask]*: Used for accessing crypto wallets & signing transactions.
- *[Metaplex]*: Protocol and SDK for creating NFTs.
- *[@solana/web3.js]*: Library for interacting with the Solana blockchain from JavaScript.
- *[@solana/spl-token]*: JavaScript library for working with SPL tokens on Solana.
- *[Vitest]*: Testing framework for the React frontend and other TypeScript code.
- *[Solana Devnet JSON-RPC]*: Public RPC access for testing blockchain transactions and programs.

10.3 Version Control and CI/CD

Git and GitHub will be used for version control and the project repository. GitHub Projects will organize issues and track their progress. LaTeX will be used for the project's technical documentation.

GitHub Actions will compile the LaTeX documentation automatically when relevant changes are pushed or proposed. Later, the CI pipeline will be extended to build and run tests for the Solana programs (using Anchor) and React frontend.

11 Use of AI and LLMs

Our team will use LLMs to assist with development and documentation throughout the project. We may use LLMs for searching for information, learning new technologies, prototyping and writing temporary code, generating tests, and assisting with documentation. Team members may use tools such as OpenAI Codex or Claude depending on their individual preferences and available subscriptions. All LLM-generated code and documentation will be reviewed by team members for correctness and relevance before being used. Code will also be verified through testing, CI/CD pipelines, and code reviews.

12 Coding Standard

The team will adhere to the coding standards outlined in Table 7.

Category	Details
React Frontend	React code will follow the official React documentation and established JavaScript/TypeScript conventions. Code will be formatted using `Prettier` and linted using `ESLint`.
Rust Backend	Rust code will follow the official Rust API Guidelines and community style conventions, formatted using `rustfmt` and linted with `clippy`.
Rust Documentation	Rust functions, structs, and modules will use `rustdoc` doc comments (`///` and `//!`) following the conventions in the official Rust documentation guide.
Docker	Dockerfiles and container images will follow Docker's official best practices for maintainability, efficiency, and security.
Git	Git will be used for version control, with a separate branch for each task and the naming conventions in Section 7.
Code Review	Changes will be submitted through GitHub pull requests and reviewed by another team member before being merged into the main branch. See the GitHub pull request review guidelines.

Table 7: Coding Standards

13 Appendix — Generative AI Use Disclosure

This section will provide an overview of how AI tools were used in the making of this document.

13.1 AI Tools Used

- Claude Sonnet 5.
- GPT-5.6 Luna.
- Gemini 1.5.

13.2 How and Where Used

Some members used AI to aid in explaining certain machine learning topics and applications, as there were varying levels of experience in the subject matter.

Additionally, AI was used to facilitate some brainstorming. This occurred in parts of the Team Charter as supplementary ideas for elements of a code of conduct and ways to reward members for positive contributions. It was also used in the Expected Technology section to generate a list of potential libraries that could be used in the project. The AI models generated lists of ideas for each of these tasks, and aided in offering a basis for further research into available technologies and team charter ideas. Everything produced by the AI models were reviewed and verified (see the verification section below) before being incorporated into the document.

AI tools were also used throughout this document to check for grammar and spelling errors, but were not used to write whole sentences or significantly rewrite text that had been written by members. Occasionally, AI tools were also used to learn LaTeX syntax (how to insert a hyperlink, for example).

13.3 Verification of AI-Generated Content

AI explanations of complicated subject matter were used together with reliable sources, such as the Solana Handbook.

The instances in which AI was used for brainstorming related to the team charter were purely for basic idea generation, and it was up to the discretion and opinion of the group to deem them valuable to the project. The list of potential libraries that was generated was used to build awareness of existing tools. The decision to include specific libraries was made based on further research into the official documentation of the libraries, not the AI response.

When AI tools were used to check grammar and spelling, team members still critically reviewed the document in its entirety to ensure the document was still understandable and to double check the grammar and spelling.

Appendix — Reflection

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. **Why is it important to create a development plan prior to starting the project?**

   Creating a development plan prior to starting the project helps to encourage thoughtful communication between team members and facilitates helpful discussions regarding expectations for the individual and for the group.

   It helped to ensure that we were all on the same page and helped us to think about the future of the project. Outlining the Proof of Concept plan helped us to confront what is reasonable within the scope of the project, and we were able to identify possible pain points. For example, we realized that the UI will most likely be a large part of the project, but is not necessary to be super involved. In contrast, we found that we are still shaky on how we will implement the reputation system. Thus, this helped us to narrow our focus and identify the expected pain points of the initial development of the system.

   Outlining roles and assigning actual metrics to the expectations really helped with administrative organization and getting a better understanding of how we want to work both as a group and individually. For example, conflict resolution was not something we had considered. However, we have now developed a system that will hopefully help in catching issues early, hold everyone accountable, and address the issues in a respectful, agreed-upon manner. It also helped us to think about the necessity of having a plan for consistent contributions and to have deadlines throughout the project to keep the team on track and avoid major crams.

   While we understand that the information laid out in this development plan may not end up going completely as expected, creating this document led to fruitful discussions and agreements, and helped us to get a better understanding of the project as a whole.

2. **In your opinion, what are the advantages and disadvantages of using CI/CD?**

   CI/CD can be extremely helpful, especially in group projects, when executed correctly. It can help to outline certain standards. For example, having a set of rules that must be adhered to in order for a pull request to be accepted.

   It can also help to prevent bugs from unknowingly arising when changes are made to existing code and disrupting code that was previously written off as meeting requirements and passing test cases. This is especially useful in a project with many collaborators and on complex projects, as small bugs introduced can be hard to catch and to pinpoint when they were introduced.

   However, CI/CD can be complicated to implement and can add to project overhead.

   As a group, we have decided to implement guidelines that must be followed when merging a branch into Main (no conflicts, at least one reviewer), the existing automatic PDF generator (with potential modifications as the project progresses), and the addition of unit testing where deemed necessary. We may choose to add more CI/CD elements to the project in the future, at the discretion of the group.

3. **What disagreements did your group have in this deliverable, if any, and how did you resolve them?**

   There were no disagreements in this deliverable. Everyone was respectful in sharing their ideas and listening to others. Any misunderstandings that were voiced were met with non-judgmental education, and agreements were decided from there. For example, there was uncertainty regarding the feasibility and expectations for the Proof of Concept. This led to educational discussions, constructive criticism, and educational moments.

   The team navigated these discussions with respect and open-mindedness, and we believe this bodes well for the future of the project, conflict resolution, and the team dynamic.

Appendix — Team Charter

External Goals

For this Capstone, our team hopes to deliver a project that is both challenging and enjoyable to create. We aim to build something beyond simply fulfilling the requirements of a school project—something that we can be proud of and potentially carry forward into our professional careers. By focusing on producing a meaningful, well-developed, and high-quality project, we believe that a strong performance in the Capstone course will naturally follow.

Attendance

This section summarizes the ground rules and expectations regarding team member attendance.

Expectations

All team members are expected to attend weekly team meetings and meetings with our supervisor. Attendance at every Capstone course lecture is not required, provided that members remain up to date with the course material through either attending the lecture in person or reviewing the material independently.

Attendance at all required team and supervisor meetings will be tracked through GitHub Issues. All members are expected to maintain a 95% attendance rate throughout the project.

When a meeting is held in person, team members should make their best effort to attend in person. If a member is unable to attend in person, they should notify the team at least 5 hours in advance and attend virtually instead. If a member is unable to attend the meeting altogether, they must notify the team as soon as possible, preferably when they become aware of the conflict and at least 24 hours before the meeting when reasonably possible. Different expectations may apply depending on the circumstances.

Acceptable Excuse

Team members are expected to communicate with the rest of the team if they are unable to attend a meeting or meet a project deadline. Each member may miss up to three team meetings per term with prior notice. Acceptable reasons may include illness, family emergencies, religious observances, or previously arranged commitments that cannot reasonably be rescheduled.

If a member exceeds three absences in a term, the team will discuss the member's participation, expectations, and responsibilities. If the issue cannot be resolved within the team, the team may involve the course professor or supervisor.

Team members are expected to meet all assigned deadlines. Exceptions will generally only be made in the event of an emergency, as outlined in the following section.

In Case of Emergency

If a team member is unable to meet a deadline or complete their assigned work due to an emergency, they must communicate this to the team as soon as they become aware that the work cannot be completed on time.

The team will then discuss appropriate accommodations and, if necessary, redistribute the affected work to ensure that project deadlines can still be met. If another team member must take on the incomplete work, the original team member will be expected to take on additional responsibilities during a subsequent phase, where appropriate, to help maintain a fair distribution of workload.

Accountability and Teamwork

Quality

To reach our goal of using this capstone experience to build something that we as a team are proud of, all members agree to make contributions that meet or exceed expectations. Level 4 of the rubric of deliverables will be referenced and used as a guideline for minimum expected requirements.

Team members are also expected to come to meetings with their assigned tasks completed and prepared to share their progress. Members are expected to actively contribute to team discussion and provide constructive criticism to help augment the quality of the project.

During code reviews, reviewers are expected to provide at least one point of constructive criticism to encourage consistent improvement.

Attitude

All team members will contribute to creating a welcoming and non-judgmental environment in which members can feel comfortable openly sharing ideas. All ideas will be considered in their entirety and seen as valuable contributions to the development process, whether they are accepted or viewed as opportunities to provide constructive criticism and identify miscommunications or misunderstandings.

Members are expected to communicate respectfully and refrain from interrupting or speaking over others. Everyone should be actively listening during meetings and provide thoughtful insight to team discussions.

All members will communicate respectfully and professionally. Contributions will be acknowledged and openly appreciated. Cooperation is expected between

team members and can be showcased through a willingness to learn, to compromise, to help fellow members when reasonably possible, and to give useful feedback during code reviews.

All disagreements are expected to be handled professionally and constructively. Should conflict arise, members will take a reasonable break until they feel that they can address it in a productive and professional way. If conflict arises that cannot be resolved by the team, a meeting with the TA and professor will be requested, and the entire team is expected to attend, even if certain members are not directly involved.

Stay on Track

To ensure deadlines are consistently met and the project stays on track, a GitHub kanban project will be used and maintained to stay up to date on all issues and milestones/deliverables. All issues will be labelled with the type, the associated milestone, the assignee(s), and the target deadline for the issue to be completed and ready for review.

Deadlines will be assigned to each issue to encourage incremental, iterative, and consistent progress, rather than big pushes before externally enforced deliverable deadlines. The schedule and availability of members will be taken into consideration when assigning tasks and deadlines, to ensure that tasks and timelines are manageable and do not cause unnecessary stress. Tasks will also be assigned with a goal of sharing an even workload amongst all team members (1/4 of the expected workload). Team members will be free to voice their concerns if the workload does not seem even, and the task assignments will be reviewed and reassessed.

Members should update the status of their issues regularly as the status changes. When an issue is ready for review, the assignee will create a pull request and assign a primary reviewer. The primary reviewer must respond and give feedback within 48 hours. Team member contributions will be assessed based on Git activity such as commits, pull requests, and issues completed to show consistent contributions that meet quality standards, as well as meeting the planned deadlines.

At weekly team meetings, members will be expected to share the status of what they have been working on that week, as well as action items for what they intend to work on or complete before the next meeting. Each member should have an attendance record of $> 95\%$. If consistent lateness becomes an issue, it will be addressed by the team.

In the case of the anticipation of a missed deadline, members should raise concerns about meeting deadlines at least 48 hours before the deadline to ensure the team can adjust and plan for the work to still be completed. In the case of a member needing to offload their assigned tasks onto other members to meet the deadline, it will be handled as a missed deadline and will be handled according

to what has been outlined above in **Attendance**.

Members who are not meeting expectations will be respectfully confronted at the following team meeting. They will have the opportunity to explain their situation and must come up with a detailed plan describing how they will get back on track. This plan will be reviewed and approved by the other team members, and their progress will be reviewed at each subsequent team meeting until the member consistently meets expectations. If at the following meetings it becomes clear that little to no effort is being made to get back on track, the issues will be brought to the attention of the TA in an effort to seek external support in conflict resolution.

Members who consistently meet or exceed expectations will be recognized by the team for their contributions. This may include informal recognition during team meetings, opportunities to take on preferred tasks, or small team rewards after completing major milestones. If the entire team reaches a milestone ahead of schedule while maintaining the expected quality of work, the team may use the additional time for a team social activity or other agreed-upon reward.

Team Building

Team building activities will be planned and hosted biweekly. Potential events may include, but are not limited to, team lunches and trivia nights. The events will be planned for a time that works for all members' schedules and will be mindfully inclusive of the needs and interests of all team members.

Decision Making

Team decisions will occur through consensus, first allowing all team members to voice their opinions and concerns. The team will collectively come to an agreement, with the success of the team and project as a whole as a priority.

In the case of a disagreement, each member will be treated with respect and given the opportunity to express their opinions honestly and respectfully. Efforts will be made to come to a healthy compromise.

If time constraints allow, any disagreement that cannot be resolved within the meeting will be revisited at a subsequent meeting. All members of the team will take time between meetings to view the situation from a different perspective to encourage empathy in the decision-making process and academic research, if necessary.